Un modèle simple de maintenance

Simulateur d'endommagement d'une flotte de composants.
Évaluation des politiques de maintenance.

Équipe M2D

I . The Simulator

A. Cases/Dataset
B. KPIs & Cost Function
C. Mathematical Description

II. Experiments / Analysis

A. Exploratory Analysis
B. Parameter Impacts
C. Model Inversion
D. Policy Optimization

III. Maintenance Pipeline

A. Pipeline Library
B. User Interface

I. The Simulator (SimMCM2D)

I. Simulator (SimMCM2D)

Implementation:
- Python Package
- PIP Installable

Functionalities:
- *sample_datasets*
- *get_kpi*
- *sample_to_table*

Documentation:
- README.md - Installation/Requirements
- examples/simulator_distributions.ipynb
- examples/simulator_example.ipynb

Simulator (SimMCM2D)

Dataset Structure:

- *system_id* / *component_id*
- *event_date* / *event_time*
- *event_type*:
  - inspection, replacement, pane
- *event_report*:
  - inspection reports (expired or wornout);
  - replacement reports (unused or reused);
- *system_age* / *component_age*
- *usage_since_last_event_h*
- *FF* (failure found)
- cost_event / cost_cumulated

Simulator (SimMCM2D)

Saving to Outputs Files

One can call directly the function `sample_to_table`, responsible for saving the sample and relevant parameters to a table. One can also use the functionality `.to_csv` from pandas to save the generated dataset to a file.

[15]

```python
df.to_csv("sample_system_dataset_0.csv")
```

Python

[16]

```python
sample_to_table(df, output_table_filepath="sample_system_datatable.csv",
                param=param,
                costs=costs,
                n_systems=250,
                date_first = "2010-01-01",
                date_final = "2025-12-31 12:00",
                id_0_component=0,
                id_0_system=0,
                output_data_filepath="sample_system_dataset_0.csv")
```

Python

Alternativelly, the function `sample_datasets` can include the fields:

- `output_data_filepath` - used to save the dataframe generated
- `output_table_filepath` - used to save/reunite the parameters used

Simulator (SimMCM2D)

	system_id	component_id	event_date	event_time	event_type	event_report	system_age	component_age	usage_since_last_event_h	FF	cost_event	cost_cumulated
0	1	1	2010-01-08	20:35:21	inspection	NaN	188.59	188.59	188.59	NaN	100	100
1	1	1	2010-01-15	15:12:59	inspection	NaN	351.22	351.22	162.63	NaN	100	200
2	1	1	2010-01-21	21:12:00	inspection	NaN	501.20	501.20	149.98	NaN	100	300
3	1	1	2010-01-23	15:20:16	failure	NaN	543.34	543.34	42.14	NaN	1200	1500
4	1	1	2010-01-23	15:20:16	replacement	unused	543.34	543.34	0.00	NaN	1100	2600
5	1	2	2010-01-29	09:54:39	inspection	NaN	681.91	138.57	138.57	NaN	100	2700
6	1	2	2010-02-04	18:07:06	inspection	NaN	834.12	290.78	152.21	NaN	100	2800
7	1	2	2010-02-10	20:52:03	inspection	NaN	980.87	437.53	146.75	NaN	100	2900
8	1	2	2010-02-18	16:05:40	inspection	NaN	1168.09	624.76	187.23	NaN	100	3000
9	1	2	2010-02-22	04:51:08	failure	NaN	1252.85	709.51	84.76	NaN	1200	4200

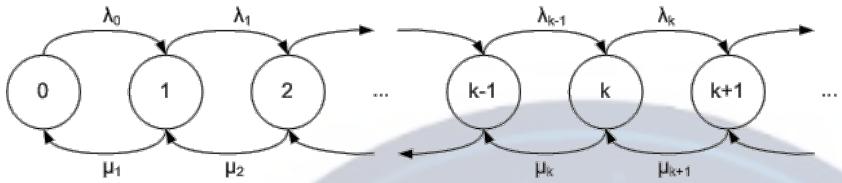
eta	beta	expiration	mu	sigma	theta	inspection_deviation	inspection_threshold	r	nu	replacement	inspection	component	failure	n_systems	date_first	date_final
720	3	792.0000000000001	168	25.2	12	0.05	0.5	0.0005	0,00005	1000	100	100	1200	250	2010-01-01	2025-12-31
720	3	792.0000000000001	168	25.2	12	0.05	0.5	0.0005	0,00005	1000	100	100	1200	100	2010-01-01	2025-12-31
720	3	792.0000000000001	168	25.2	12	0.05	0.5	0.0005	0,00005	1000	100	100	1200	250	2010-01-01	2025-12-31
720	3	792.0000000000001	168	25.2	12	0.05	0.5	0.0005	0,00005	1000	100	100	1200	100	2010-01-01	2025-12-31

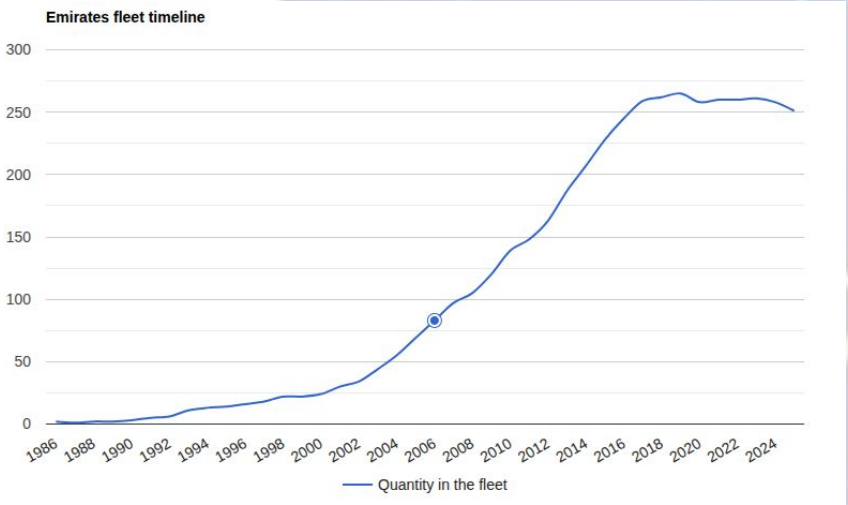
output_data_filepath	Average of failure per month	Average of failure per trimestre	Average of failure per year	Stability per month	Stability per trimestre	Stability per year	Average of rep
sample_system_dataset_0.csv	168.38378378378377	486.734375	1946.9375	111.176072437727	333.9150218475895	1345.5556953046	258.36979166
sample_system_dataset_1.csv	67.29729729729729	194.53125	778.125	42.9600660999187	131.79666484056125	530.6772991501	104.87894736
sample_system_dataset_0.csv	157.9	468.765625	1875.0625	80.6643957215892	241.42929999244348	969.1132695235	246.11979166
sample_system_dataset_1.csv	64.02083333333333	192.0625	768.25	25.5151713543303	75.24338650208128	297.2827988633	101.453125

Simulator (SimMCM2D)

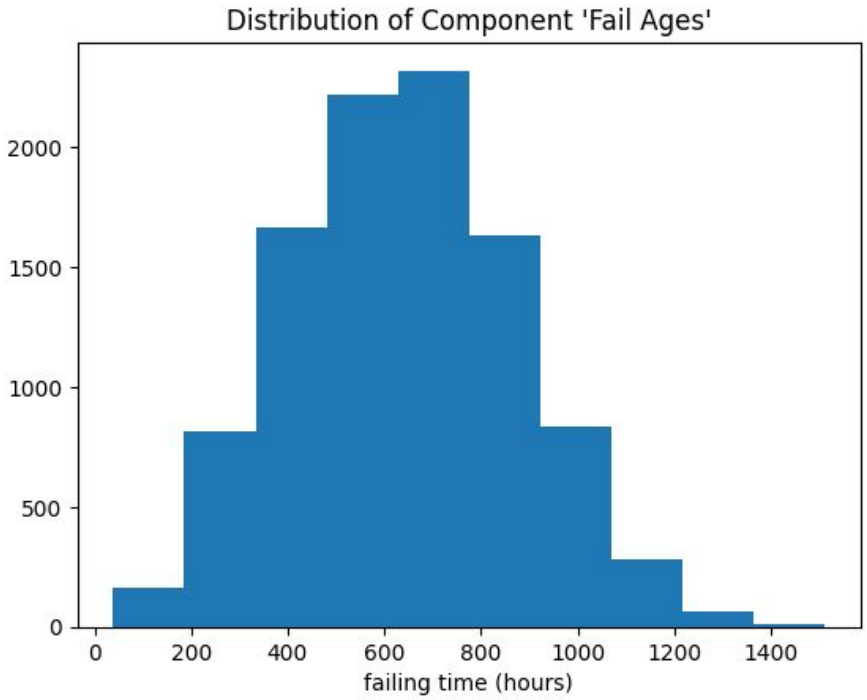
Simulated Cases:

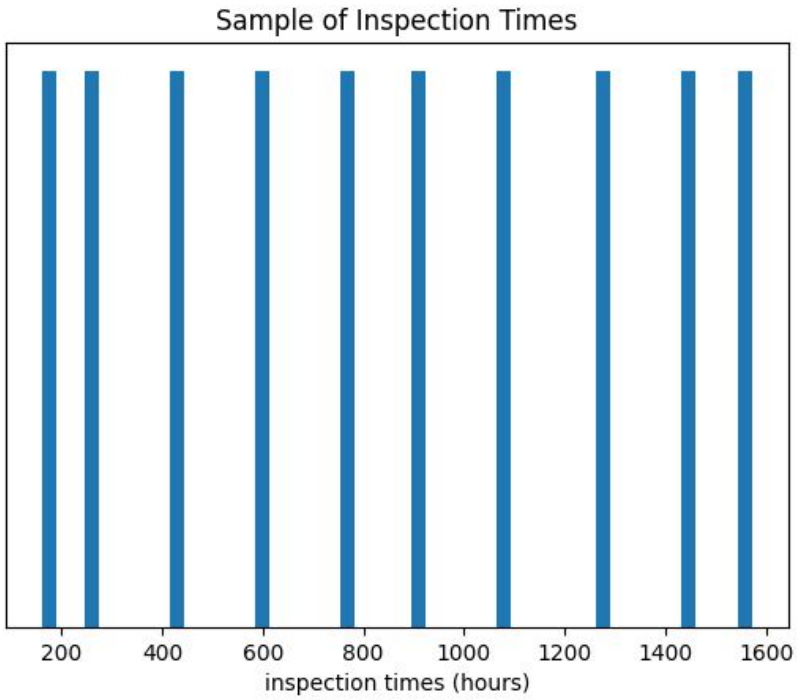
- Component Failure (Pane)
  - Following a Weibull Distribution
- Inspection Scheduling
  - Inspection without Replacement
  - Inspection Leading to Replacement
  - Uncertain Inspection Measurements
- Replacement Scheduling
  - Delay Before Replacement is Required
  - Component Reuse and Maintenance
  - Automatic Replacement at Age Limit
- Fleet Evolution Over Time
  - Following a Birth-Death Process



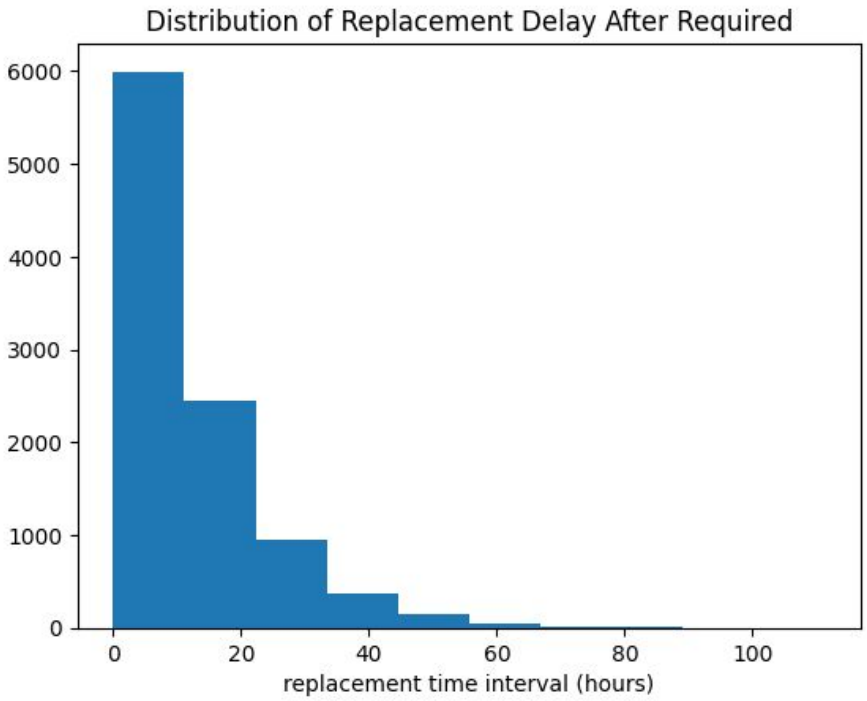


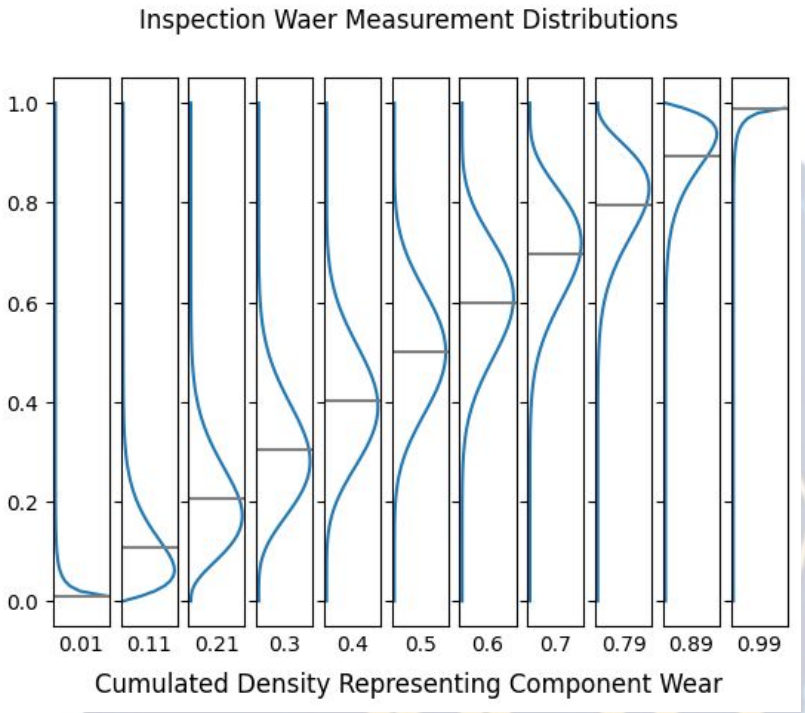
Simulator (SimMCM2D)



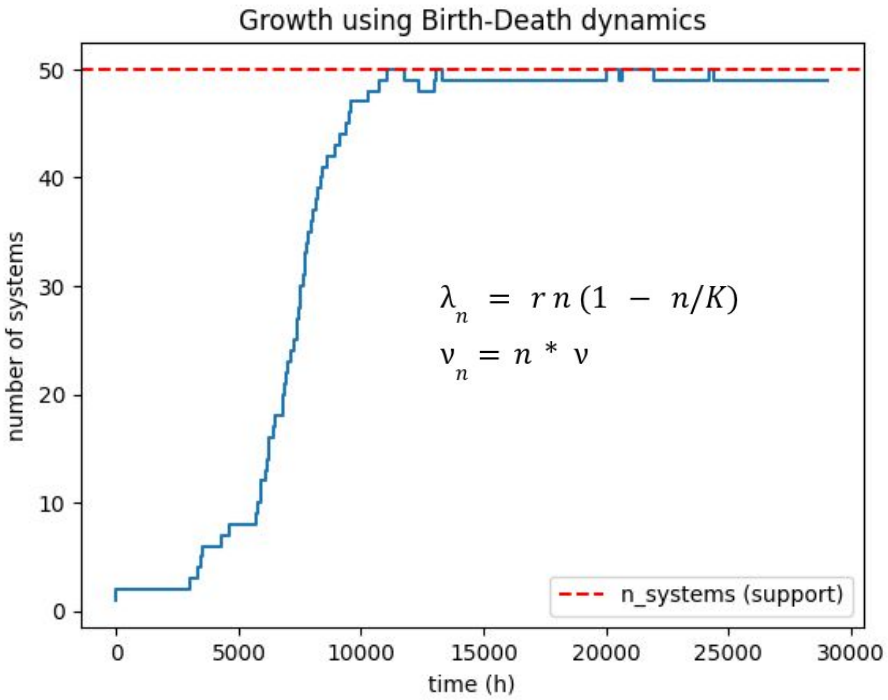


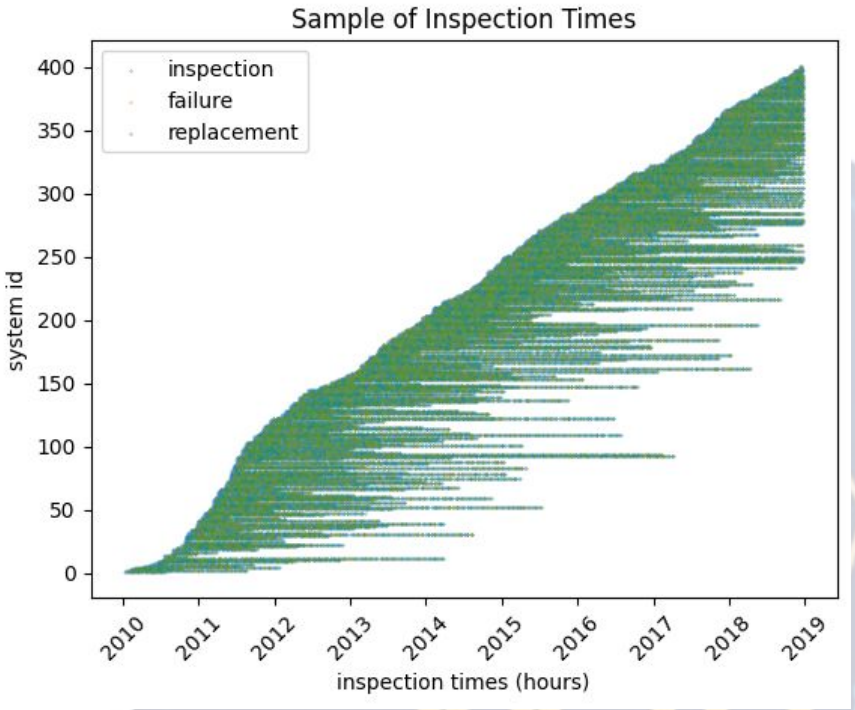
Simulator (SimMCM2D)





Simulator (SimMCM2D)





I.B. KPI Key Performance Indicator

Definition:

- Quantitative measure
- Evaluate effectiveness & success
- Follow reliability of the equipments
- Cost optimisation
- Track progress over time

=> decision support

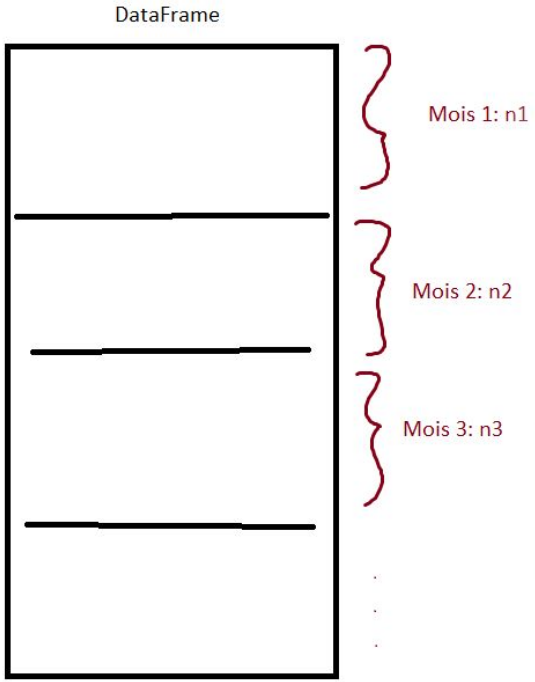
3 categories:

- number of failure & replacement
- Quality of the detector
- Costs

I. B. KPI: Number of failure & replacement

Month (m) Quarter (t) Year (y)

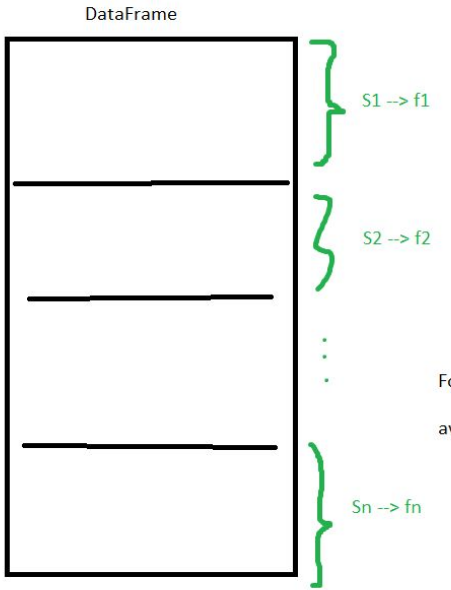
- For the Fleet
  - ex: per month



$$\frac{n1 + n2 + n3 + nm}{nm} = avg_fail_m$$

- Per System Metrics (Reliability)

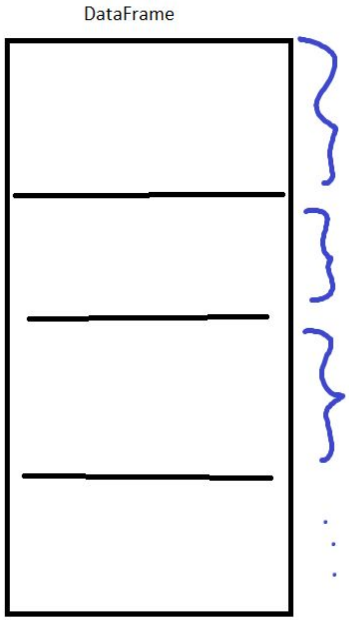
- **avg_fail_syst_total:** Avg over the total lifetime of the fleet



For a fleet with n systems:

$$\text{avg_fail_syst_total} = \frac{f1 + \ldots + fn}{n}$$

- **avg_fail_syst_:** Avg per system and per period



For a fleet with n systems observed over m months:

$$\text{avg_fail_syst_m} = \frac{M1S1 + M1S2 + \ldots + M1Sn + \ldots + MmS1 + \ldots + MmSn}{n.m}$$

- PER = replacement / failure
  - > 1 : over-maintenance
  - < 1 : under-maintenance
  - ≈ 1 : balance between planned interventions and observed failures.

KPI : Quality of the detector

- $Precision = \frac{Detection}{Detection + False\ alerts}$

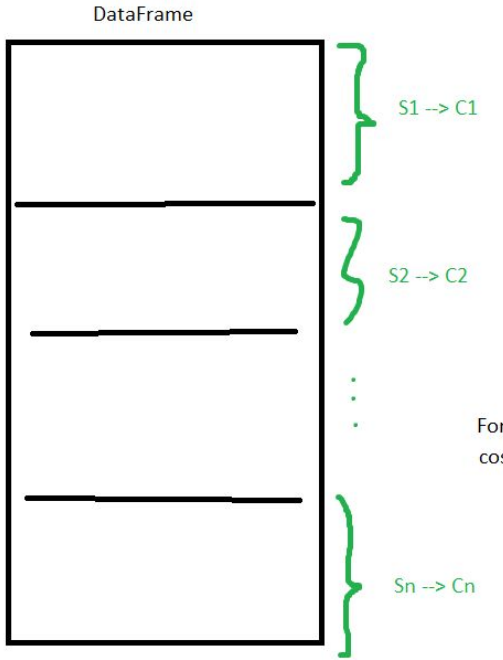
  → = 1 : Every intervention is useful.

- $Recall = \frac{Detection}{Total\ nb\ of\ failures}$

  → = 0.5: one failure out of two is detected by the system

KPI : Costs

- **cost_per_syst:** average cost of a system across the entire fleet

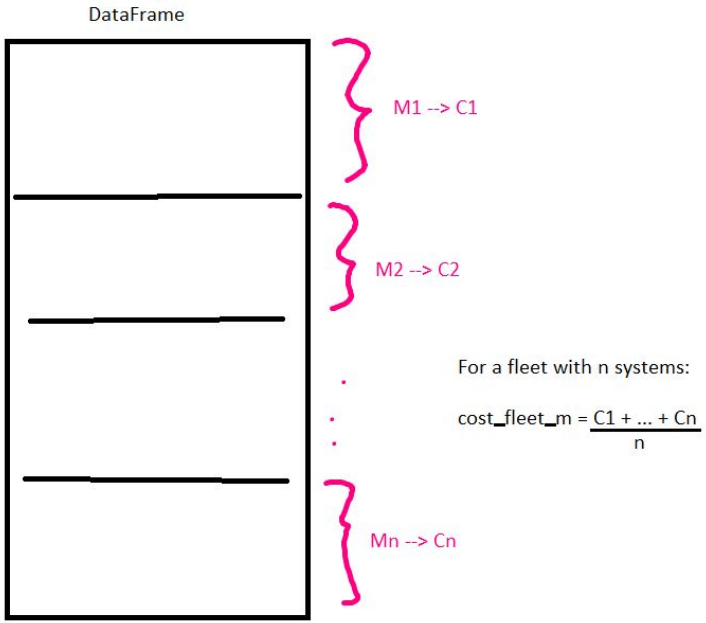


For a fleet with n systems:

$$\text{cost_per_syst} = \frac{C1 + \ldots + Cn}{n}$$

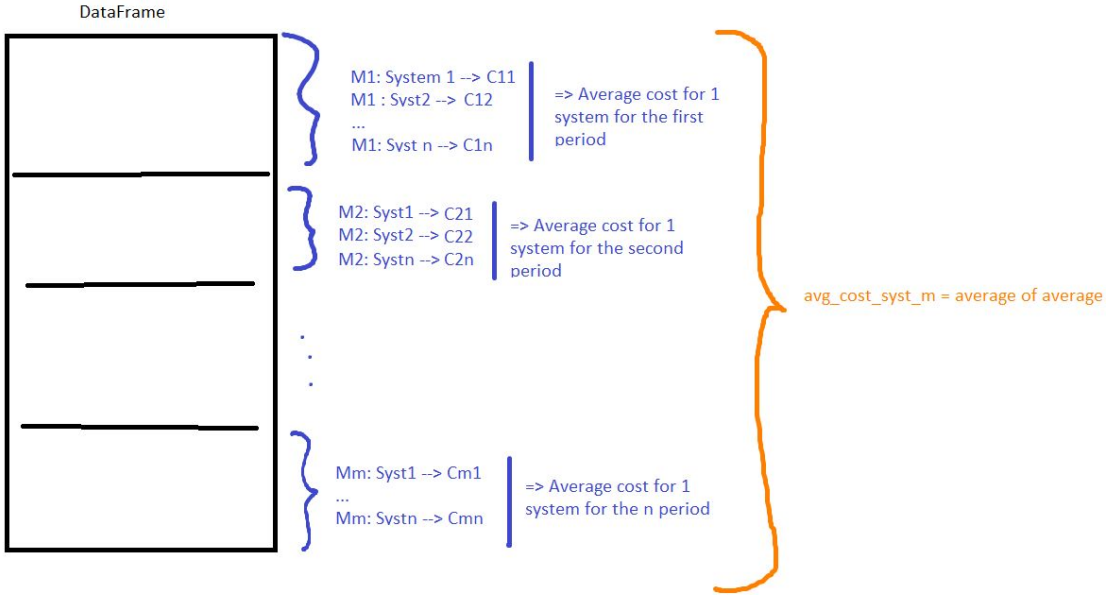
- **cost_fleet_:** cost of the entire fleet per period

- **avg_cost_syst_ :** average cost of a system per period



For a fleet with n systems:

$$cost_fleet_m = \frac{C1 + \ldots + Cn}{n}$$



I. C. Mathematical Description

Modéliser la dégradation : Loi de Weibull

Le temps avant la panne T suit une distribution de Weibull :

$$T \sim \text{Weibull}(\eta, \beta)$$

Paramètres physiques :

- η (**Échelle**) : Ordre de grandeur de la durée de vie.
- β (**Forme**) : Caractérise le type de vieillissement.

Fonctions clés :

- Répartition : $F(t) = 1 - \exp\left(-(t/\eta)^{\beta}\right)$
- Survie : $S(t) = \exp\left(-(t/\eta)^{\beta}\right)$

Choix du simulateur : $\beta = 3$. Le taux de risque augmente avec le temps (phénomène d'usure progressive).

Calendrier stochastique des inspections

Les inspections ne sont pas strictement périodiques pour refléter les contraintes opérationnelles.

Les intervalles entre contrôles suivent une **Loi Normale** :

$$\Delta I \sim \mathcal{N}(\mu, \sigma^2)$$

L'instant de la k-ième inspection est :

$$I_k = \sum_{j=1}^{k} \Delta I_j$$

Paramétrage retenu pour la simulation :

- Période moyenne $\mu = 168$ h (1 semaine).
- Variabilité $\sigma = 15\%$ de μ.

Modéliser l'inspection imparfaite : Loi Beta

L'usure réelle p est une **variable latente** définie par la fonction de répartition de Weibull : $p = F(t) \in (0, 1)$.

La mesure observée W est bruitée :

$$W \mid p \sim \mathrm{Beta}(a, b)$$

Contrainte d'espérance (la mesure est en moyenne correcte) :

$$\mathbb{E}[W] = \frac{a}{a+b} = p$$

Paramètre de confiance κ (on pose $\kappa = a + b$) :

- $a = \kappa p \quad \text{et} \quad b = \kappa(1-p)$
- **Variance** : $\mathrm{Var}(W) = \frac{p(1-p)}{\kappa+1}$

Plus κ est élevé, plus le capteur est précis.

La dynamique d'inversion

Sens direct (Génération du simulateur) :

$$(p, \kappa) \xrightarrow{\text{analytique}} (a, b) \xrightarrow{\text{aléatoire}} W$$

Sens inverse (Estimation empirique) :
Comment retrouver la qualité κ à partir des observations W_i ?

Utilisation de la **méthode des moments** sur l'échantillon :

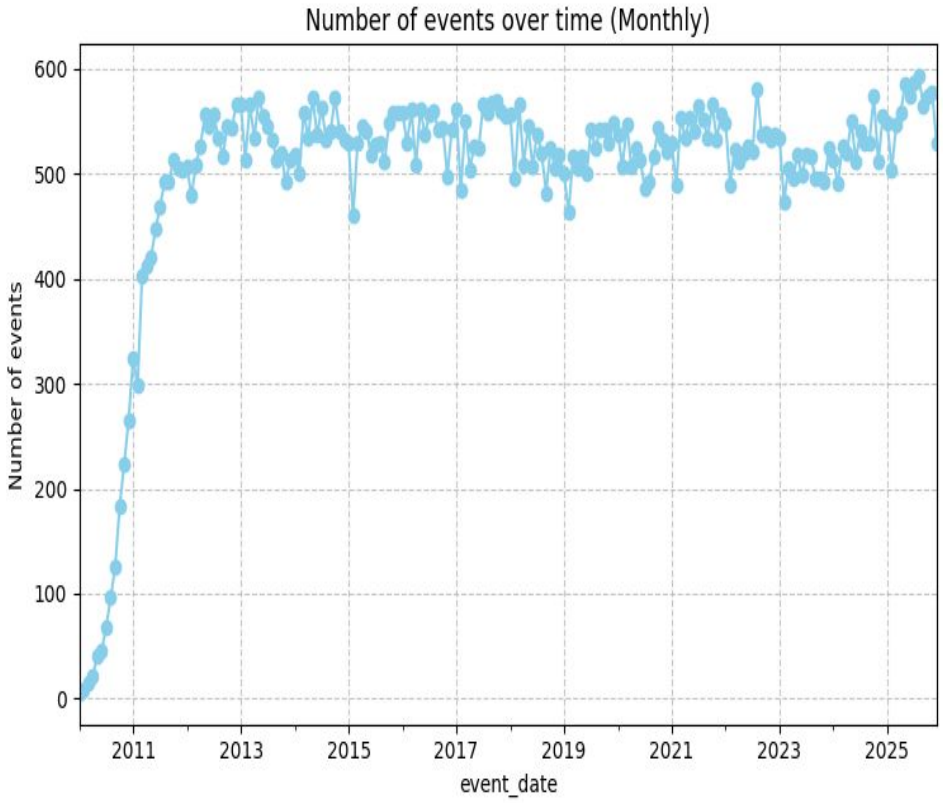
1. Estimation de la moyenne $\bar{W}$ et de la variance s^2.
2. Inversion mathématique :

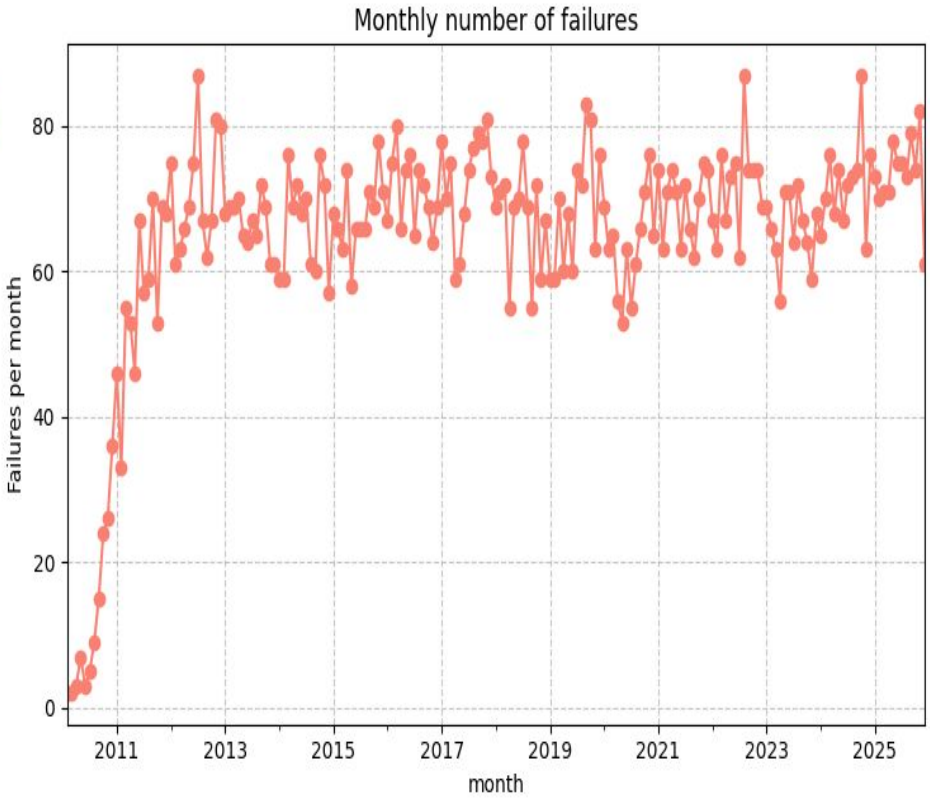
$$\hat{\kappa} = \frac{\bar{W}(1 - \bar{W})}{s^2} - 1$$

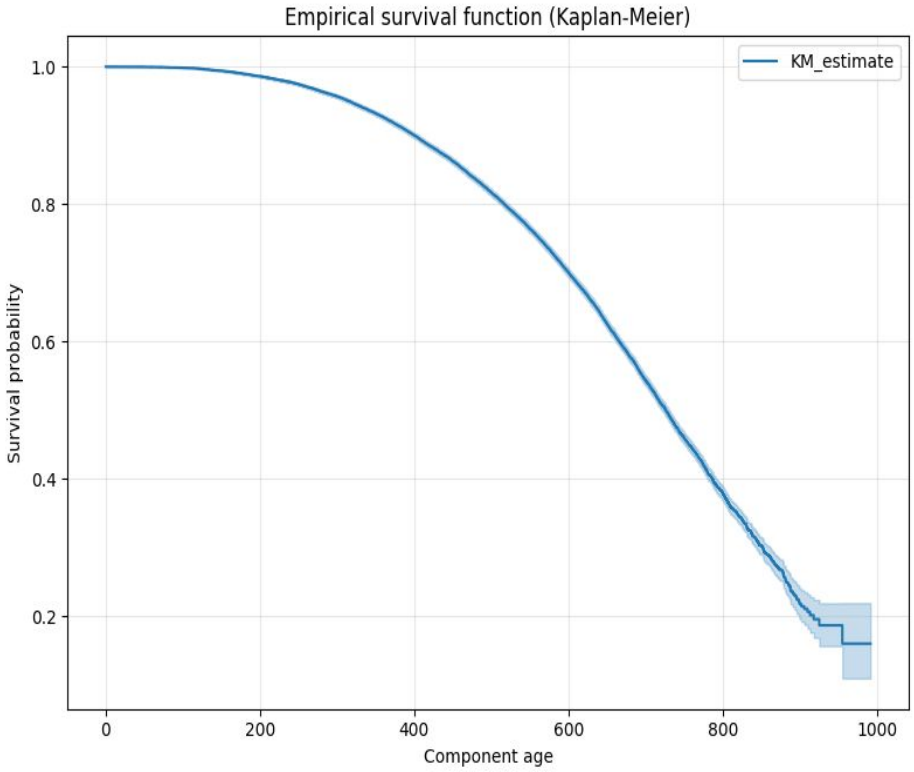
3. Déduction de $\hat{a} = \hat{\kappa}\bar{W}$ et $\hat{b} = \hat{\kappa}(1 - \bar{W})$.

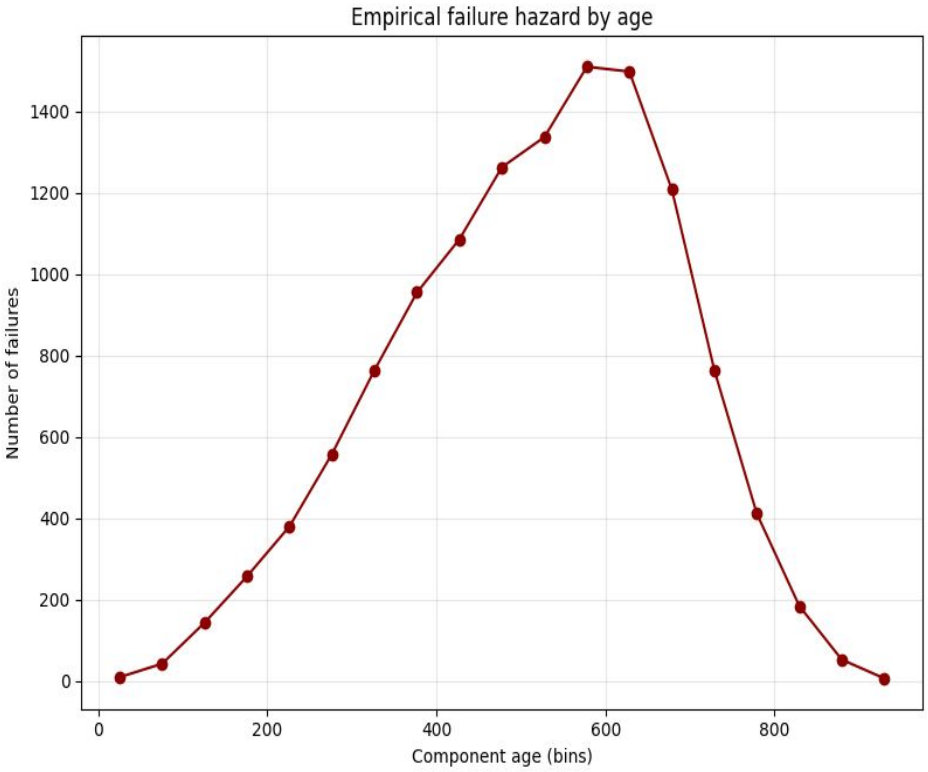
II. Experiments

A. Exploratory Analysis






Empirical survival function (Kaplan-Meier)
KM_estimate
Survival probability
Component age
1.0
0.8
0.6
0.4
0.2
0
200
400
600
800
1000


Empirical failure hazard by age
Number of failures
Component age (bins)
1400
1200
1000
800
600
400
200
0
0
200
400
600
800

II. B. Impact des paramètres sur les performances du système

Objectif

Étudier l'influence des paramètres du modèle sur les performances d'un système de maintenance simulé.

Méthode

- Simulation Monte Carlo sur **200 systèmes**
- Horizon de simulation : **1 an (8760 h)**
- **25 répétitions** par configuration
- Analyse de sensibilité :
  - variation **d'un paramètre (1D)**
  - variation **de deux paramètres (2D)**

Indicateurs étudiés

- coût moyen de maintenance
- nombre de pannes
- précision et rappel de détection
- efficacité préventive (PER)

Ces indicateurs permettent d'identifier les **paramètres les plus influents** et les **compromis entre coût et performance**.

Influence du seuil de détection

Le **seuil de détection** contrôle la sensibilité du système lors des inspections.

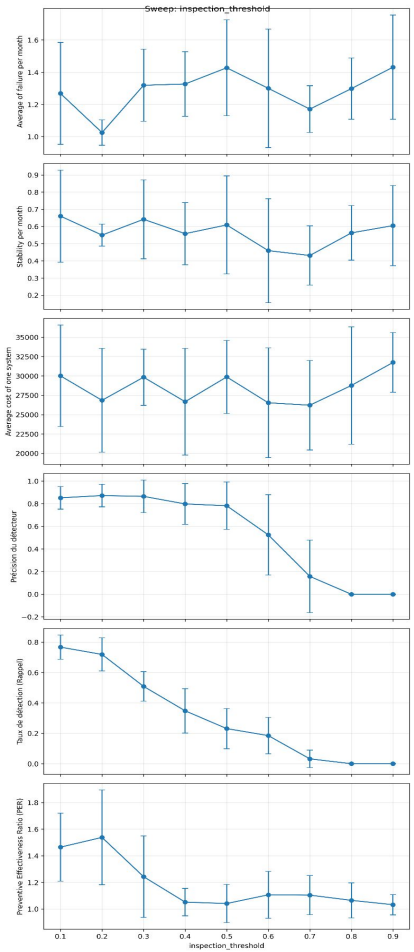
Résultats principaux

- Lorsque le seuil augmente :
  - **la précision chute fortement**
  - **le rappel diminue**
  - **le nombre de pannes augmente**
  - **l'efficacité préventive (PER) diminue**

➡ Un seuil trop élevé retarde les interventions et augmente le risque de panne.

Zone favorable observée :

threshold ≈ 0.2−0.4



Influence de la fréquence des inspections

Le paramètre **μ** représente l'intervalle moyen entre deux inspections.

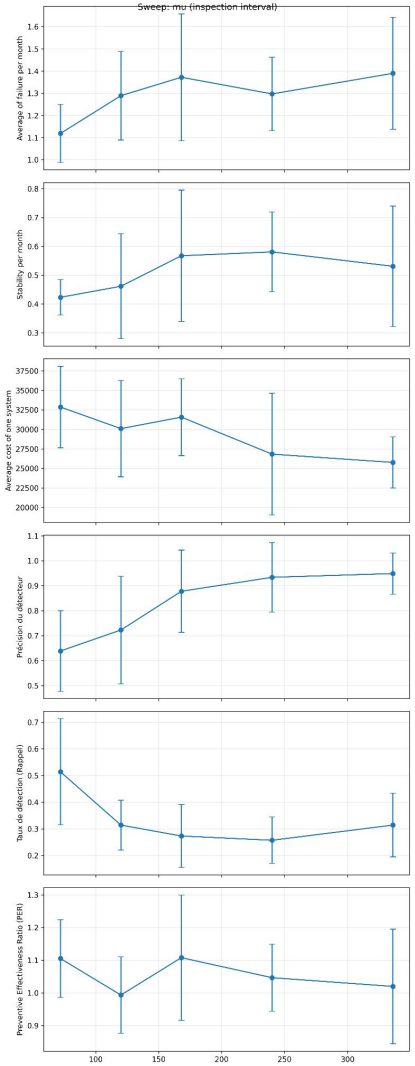
Effets observés :

- **inspections fréquentes (μ faible)**
  - coût élevé
  - davantage d'actions de maintenance
- **inspections plus espacées (μ élevé)**
  - coût plus faible
  - précision plus élevée
  - mais **rappel plus faible**

➡ Il existe donc un **compromis entre coût et qualité de détection**.

Zones intéressantes

- **120–168 h** : peu de pannes
- **240–336 h** : coût plus faible



Variabilité: inspections et dégradation

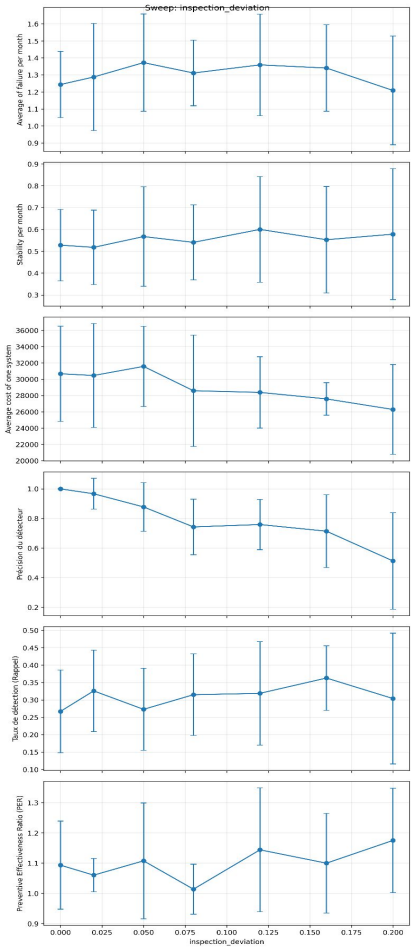
Deux paramètres de variabilité ont été étudiés :

- **inspection_deviation** : irrégularité des inspections
- **σ (sigma)** : variabilité du processus de dégradation

Résultats principaux

- Une **variabilité plus forte diminue la précision du détecteur**.
- Les inspections irrégulières rendent l'état du système **moins observable**.
- L'augmentation de σ rend également la détection **moins fiable**.

➡ Les meilleurs résultats sont obtenus pour **des niveaux de variabilité faibles à modérés**.



Interactions entre paramètres

Les analyses 2D montrent que les paramètres **interagissent fortement**.

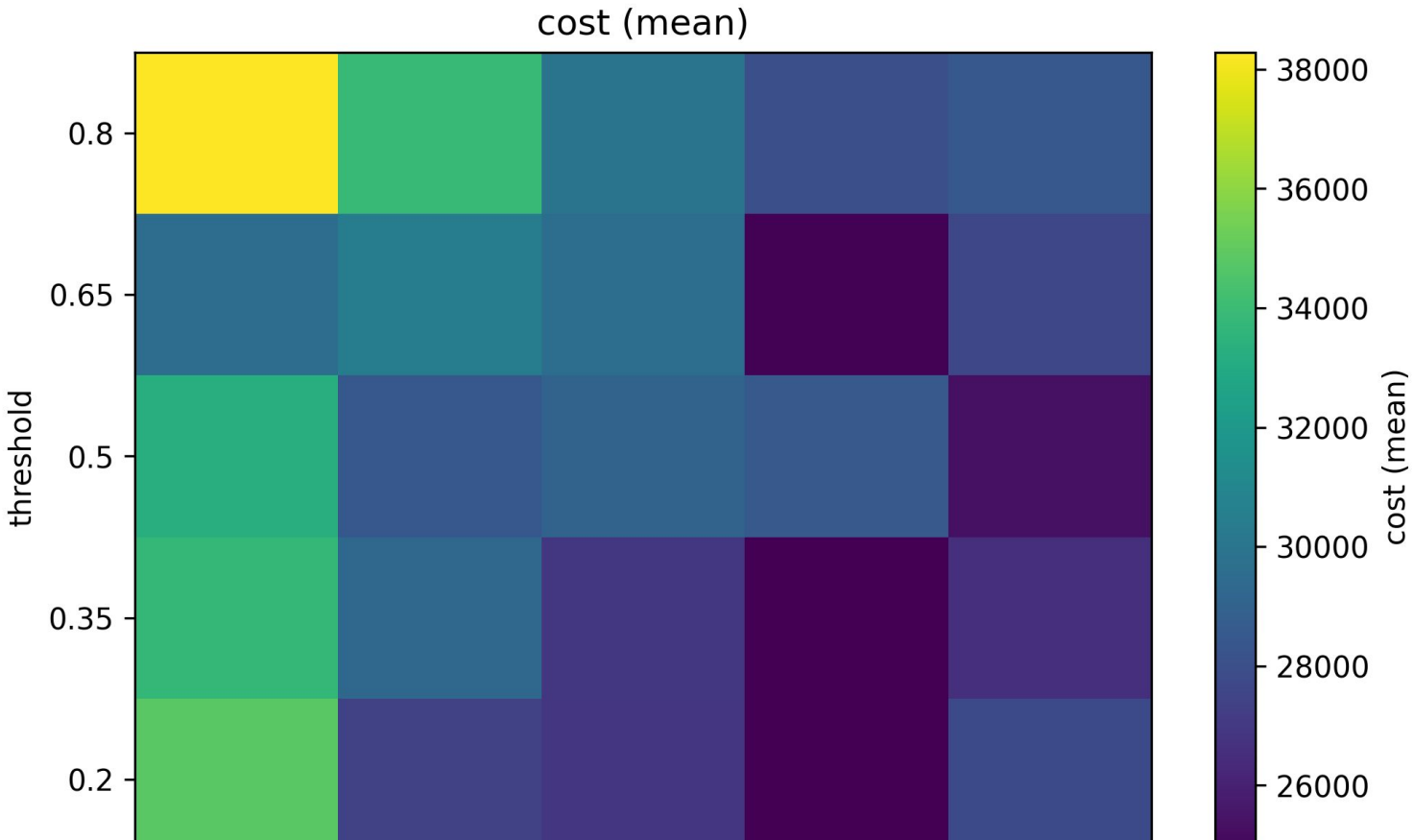
Exemple : seuil de détection × intervalle d'inspection

Résultats observés :

- coût minimal pour
  seuil intermédiaire + inspections espacées
- plus de pannes lorsque :
  - seuil élevé
  - inspections espacées

➡ Une politique de maintenance efficace doit **optimiser les paramètres conjointement** et non indépendamment.

Cout selon seuit et mu (h)



Panne selon un seuil et mu (h)

Résumé

Principaux résultats

- Le **seuil de détection** est le paramètre le plus influent :
  un seuil trop élevé réduit fortement la précision et augmente les pannes.
- L'**intervalle entre inspections (μ)** introduit un **compromis entre coût et performance** :
  - inspections fréquentes → coût élevé
  - inspections espacées → détection moins efficace.
- Les paramètres de **variabilité** (inspection et dégradation) affectent surtout la **robustesse de la détection**.
- Les analyses 2D montrent des **interactions importantes entre paramètres**.

Conclusion

➡ Le réglage optimal d'une politique de maintenance nécessite **une optimisation conjointe des paramètres** afin d'équilibrer :

- coût
- risque de panne
- qualité de détection

II. C. Model Inversion

Estimation (1) - Le calendrier

Objectif : Retrouver μ et σ depuis le fichier CSV.

Extraction de la suite d'intervalles $\Delta I_1, \ldots, \Delta I_n$ (colonne `usage_since_last_event_h`).

Utilisation des **estimateurs empiriques** classiques :

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^{n} \Delta I_i$$

$$\hat{\sigma} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (\Delta I_i - \hat{\mu})^2}$$

Estimation (2) - Le défi des données

Le problème : Tous les composants ne tombent pas en panne (remplacement préventif ou limite d'âge).

On n'observe qu'une borne inférieure de leur durée de vie réelle : c'est la **censure à droite**.

Pour chaque observation i, on a un couple (t_i, δ_i) :

- $\delta_i = 1$ si la **panne** est observée.
- $\delta_i = 0$ si l'observation est **censurée**.

Estimation (3) - Maximum de Vraisemblance

Construction de la vraisemblance en séparant les deux cas :

- Si panne ($\delta_i = 1$) $\rightarrow$ contribution de la densité $f(t_i)$.
- Si censure ($\delta_i = 0$) $\rightarrow$ contribution de la survie $S(t_i)$.

Log-vraisemblance à maximiser :

$$\ell(\eta, \beta) = \sum_{i:\delta_i=1} \log f(t_i) + \sum_{i:\delta_i=0} \log S(t_i)$$

Résolution par **optimisation numérique** (ex : algorithme `scipy.minimize` en Python).

Densité et fonction de survie de la loi de Weibull

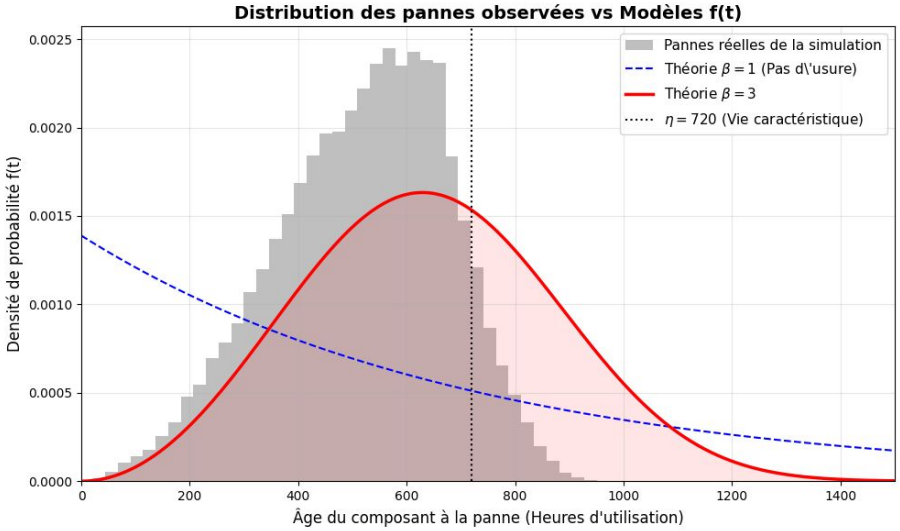
$$f(t;\eta,\beta) = \frac{\beta}{\eta}\left(\frac{t}{\eta}\right)^{\beta-1} e^{-(t/\eta)^{\beta}}$$

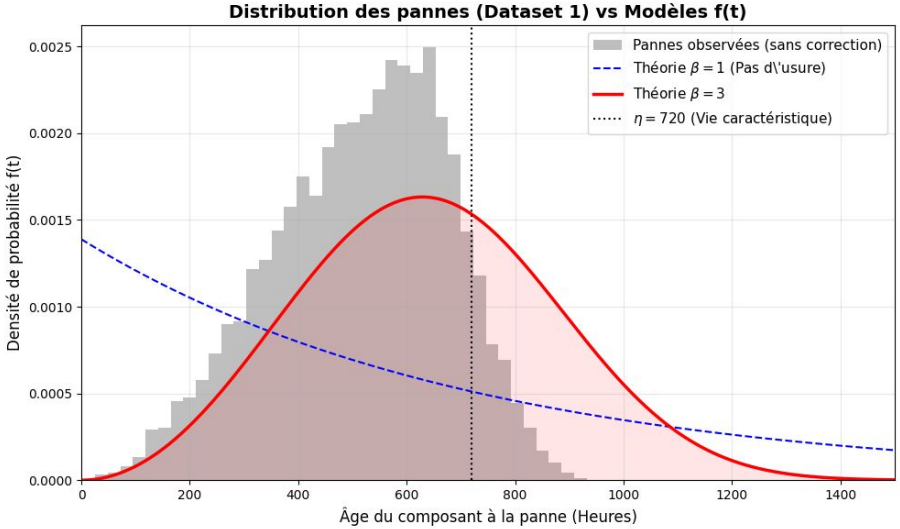
$$S(t;\eta,\beta) = e^{-(t/\eta)^{\beta}}$$

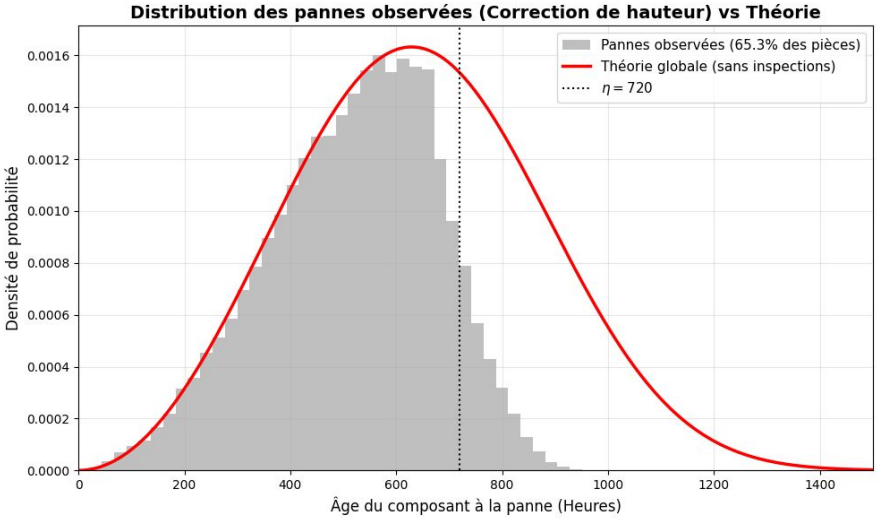
Les resultats :

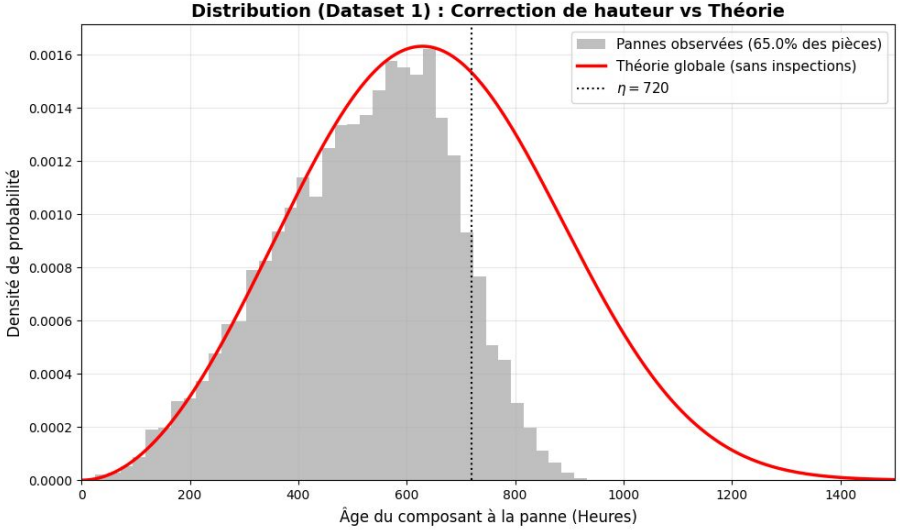
```
Analyse du fichier : sample_system_dataset_0.csv

--- 1. PARAMÈTRES D'INSPECTION (Loi Normale) ---
Mu (Moyenne)      : 166.54 heures
Sigma (Écart-type): 26.70 heures

--- 2. PARAMÈTRES DE PANNE (Loi de Weibull) ---
Eta (Échelle) : 720.16 heures
Beta (Forme)  : 3.00

--- 3. PARAMÈTRE THETA (Délai logistique) ---
Theta estimé (Moyenne du délai) : 11.35 heures

--- 4. PARAMÈTRES R ET NU (Gillespie / Naissances-Morts) ---
Heures cumulées par toutes les machines : 28,610,061 heures
Nu estimé (Mortalité) : 0.00006
r (ordre de grandeur brut) : 0.00006
```

```
Analyse du fichier : sample_system_dataset_1.csv

--- 1. PARAMÈTRES D'INSPECTION (Loi Normale) ---
Mu (Moyenne)      : 166.13 heures
Sigma (Écart-type): 27.24 heures

--- 2. PARAMÈTRES DE PANNE (Loi de Weibull) ---
Eta (Échelle) : 721.06 heures
Beta (Forme)  : 2.96

--- 3. PARAMÈTRE THETA (Délai logistique) ---
Theta estimé (Moyenne du délai) : 11.52 heures

--- 4. PARAMÈTRES R ET NU (Gillespie / Naissances-Morts) ---
Heures cumulées par toutes les machines : 11,817,261 heures
Nu estimé (Mortalité) : 0.00006
r (ordre de grandeur brut) : 0.00006
```


Distribution des pannes observées vs Modèles f(t)
Pannes réelles de la simulation
Théorie $\beta = 1$ (Pas d\'usure)
Théorie $\beta = 3$
$\eta = 720$ (Vie caractéristique)
Densité de probabilité f(t)
Âge du composant à la panne (Heures d'utilisation)
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
0
200
400
600
800
1000
1200
1400


Distribution des pannes (Dataset 1) vs Modèles f(t)
Pannes observées (sans correction)
Théorie $\beta = 1$ (Pas d\'usure)
Théorie $\beta = 3$
$\eta = 720$ (Vie caractéristique)
Densité de probabilité f(t)
Âge du composant à la panne (Heures)
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
0
200
400
600
800
1000
1200
1400


Distribution des pannes observées (Correction de hauteur) vs Théorie
Pannes observées (65.3% des pièces)
Théorie globale (sans inspections)
$\eta = 720$
Densité de probabilité
Âge du composant à la panne (Heures)
0.0000
0.0002
0.0004
0.0006
0.0008
0.0010
0.0012
0.0014
0.0016
0
200
400
600
800
1000
1200
1400


Distribution (Dataset 1) : Correction de hauteur vs Théorie
Pannes observées (65.0% des pièces)
Théorie globale (sans inspections)
$\eta = 720$
Densité de probabilité
Âge du composant à la panne (Heures)
0.0000
0.0002
0.0004
0.0006
0.0008
0.0010
0.0012
0.0014
0.0016
0
200
400
600
800
1000
1200
1400

Validation des estimateurs sur les données

Résultats obtenus sur nos datasets simulés (`sample_system_dataset`) :

Paramètre	Valeur Théorique	Estimation Empirique
Moyenne insp. (μ)	168 h	$\approx$ 167 h
Échelle Weibull (η)	30 jours (720h)	$\approx$ 722 h
Forme Weibull (β)	3.00	$\approx$ 3.00

Conclusion : Le modèle mathématique stochastique est cohérent de bout en bout, de la génération des variables latentes jusqu'à leur estimation empirique.

II. D. Parameter Optimization

Goal:
Automatically find the optimal parameters minimizing the system cost.

Parameters optimized:

$\theta = (\mu, \sigma, \theta, \text{inspection_deviation}, \text{inspection_threshold})$

Objective function:

$J(\theta) = E[C(\theta)] + \alpha \cdot \text{Std}(C(\theta))$

- Total average cost of one system
- Standard deviation

II.D. Optimization Challenges

The simulator produces a cost function that is:

- non-linear.
- non-convex.
- non-differentiable.

Therefore, no analytical gradient exists

II.D. Plan

Monte-Carlo Estimation

Estimating the cost by averaging the results of M Monte-Carlo simulations

Genetic algorithm

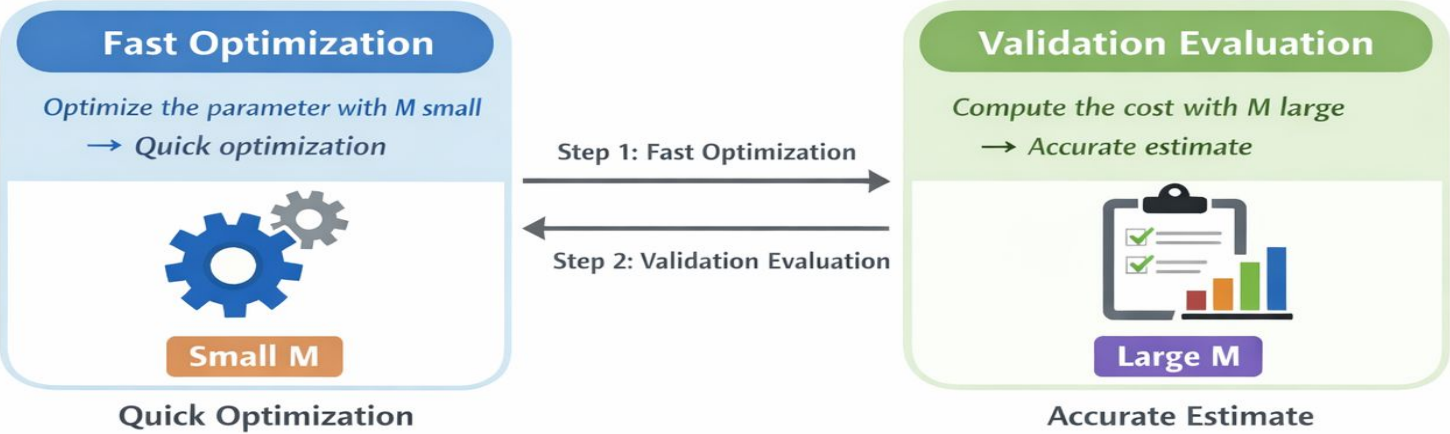
Genetic Algorithm to search for the parameters that minimize this cost.

II.D. Steps of Genetic Algorithm

1. Population initialization : Sampled uniformly within predefined bounds.
2. Fitness Evaluation
3. Selection : Choose the best among k randomly selected individuals.
4. Crossover : Combined the selected individuals.
5. Mutation : To maintain diversity, small random perturbations are applied.
6. Elitism : The best individuals are preserved across generations:
7. Immigration : some individuals are replaced by randomly generated solutions.
8. New generation :

elites + children + immigrants

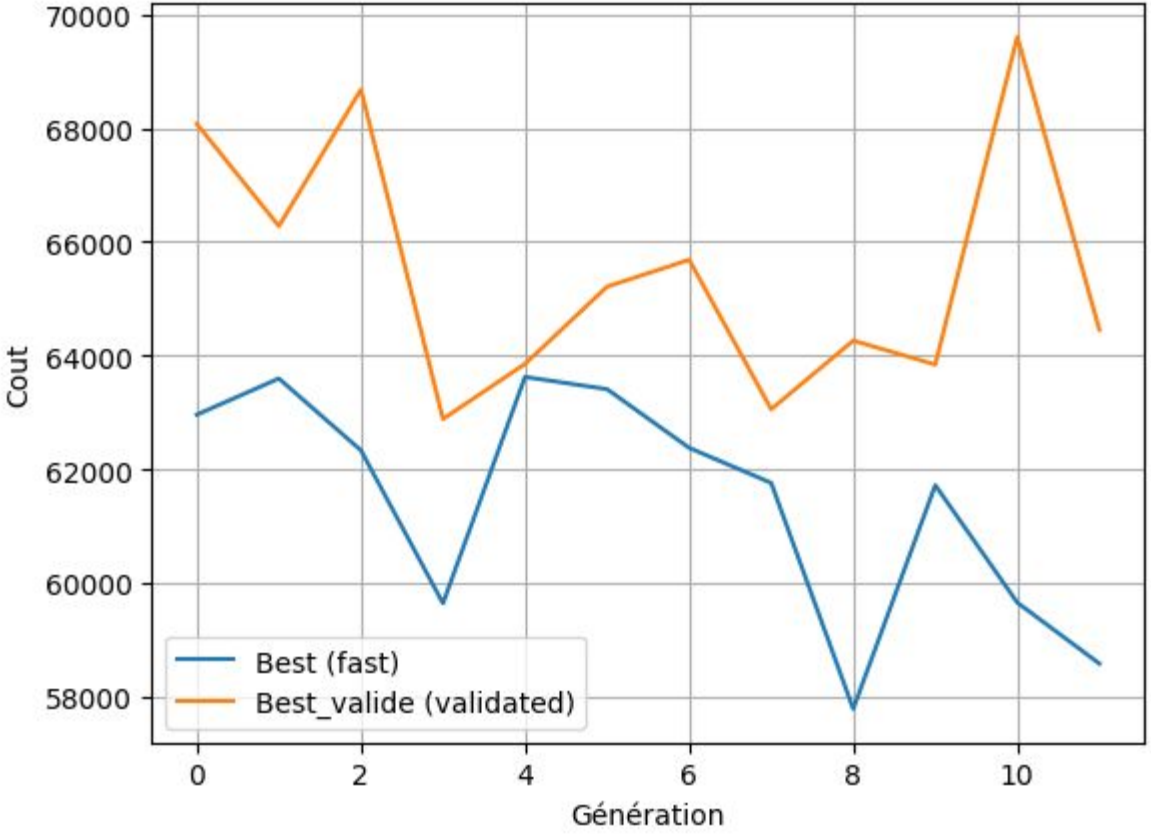
Two Stage Evaluation



GA parameters

Parameter	Value
Population size	30
Max generations	50
Elitism	2
Tournament size	4
Mutation probability	0.30
Mutation σ	0.15
M fast	2
M large	20
Immigration rate	0.2

Parameters	Estimated / Initial	Optimized
mu	166.54 (est)	304.61
sigma	26.70 (est)	56.99
theta	11.35 (est)	12.60
inspection deviation	0.05	0.9
inspection threshold	0.5	0.79



Initial cost = 75059.30
Optimized cost = 65113.49
Gain = 13.25 %

III. Maintenance Pipeline

I. A. Pipeline Library (PipMCM2D)

Functionalities:

- Parameter Estimation (unknown param)*:*
  - *estimate_parameters*
  - *estimate_parameters_filepath*
- Policy Optimization (genetic algorithm):
  - compute_cost
  - algorithme_genetique

Documentation:

- README.md - Installation/Requirements
- examples/parameter_estimator.ipynb
- examples/AG_Optimisation.ipynb
- examples/pipeline_example.ipynb

I. A. Pipeline Library (PipMCM2D)

Simulate "Real" Sample + Descriptive Analysis

→

Use "Known" Parameters + Estimate "Unknown" Parameters

→

Sample and to Optimize from "Same Family" of systems

Simulator (SimMCM2D)

Use the Optimization Functionality

We run an Optimization Policy by making use of Known and Estimated parameters:

```python
from pipmcm2d import compute_cost, algorithme_genetique
```

[ ] Python

```python
rng = np.random.default_rng(1234)

meilleur_theta, meilleur_cout, hist_best, hist_mean, hist_valid = algorithme_genetique(
    fixed_param=fixed_param, costs=costs, bornes=bornes, rng=rng,
    taille_pop=15, max_generation=30, M_test=3, M_validate=20)
```

[20] ✓ 20m 38.3s Python

```
génération |    Best_fast |    Mean_fast |   Best_valid
         1 |     62960.58 |     69864.27 |     68075.65
         2 |     63597.19 |     69468.68 |     66277.35
         3 |     62334.26 |     68821.90 |     68679.07
         4 |     59643.22 |     68264.82 |     62885.27
         5 |     63625.07 |     69399.47 |     63853.61
         6 |     63409.51 |     67252.69 |     65212.82
         7 |     62377.49 |     67036.33 |     65685.36
         8 |     61760.70 |     67207.47 |     63056.18
         9 |     57787.24 |     65788.11 |     64262.87
        10 |     61721.18 |     66699.07 |     63842.66
        11 |     59655.25 |     67523.38 |     69606.82
```

III. B. User Interface

Application overview

Streamlit

- Interface built with Streamlit (Python)
- Allows configuration of simulation parameters
- Generates datasets and KPIs
- Provides descriptive analysis and parameter estimation

Access to the Application Streamlit

- The application is accessible online:
- https://projetm2d2526-m2d.streamlit.app/
- We will now move to a live demonstration of the application.